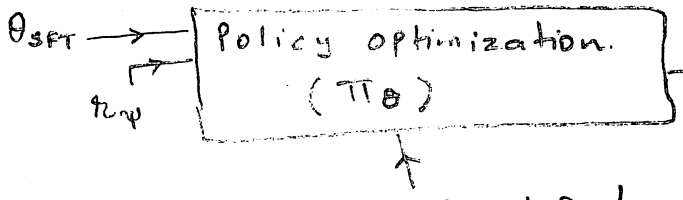
Policy optimization (RL):



Once we have trained r_ψ model that captures human preferences, we can put it in reinforcement learning (RL) framework to optimize our policy π_θ. More clearly, we have different models at this stage:

① Policy (Model) = π_θ

$\pi_\theta = \theta_{SFT}$ = Also called Actor

Role: Generates a response y for prompt x

- It will be trained.

② Reward Model: r_ψ

r_ψ as it is

Role: $r_\psi(x,y)$ + Scalar Reward (Sparse)

- Remains Frozen

③ Reference Model: π_{ref}

$\pi_{ref} = \theta_{SFT}$

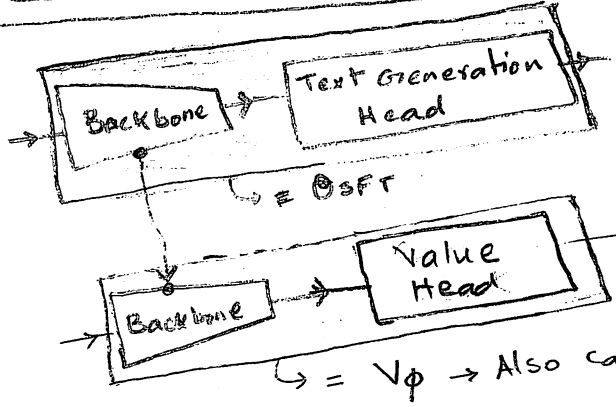
Role: Will be used to train π_θ

- Remains Frozen

④ Value Model = V_ϕ

$\theta_{SFT} = V_\phi$ ← initialize (backbones)

Expected Return from current state ← $= V_\phi(s_t)$ ← Current state



PO with RLHF: Treat LM policy π_θ as an agent that selects next tokens to:

1- maximize expected cumulative reward from r_ψ

2- Stays close to SFT-reference π_{ref} via a KL-regularizer

3- gets its updates through advantages, computed by a (separate) critic.

Some Glossary:

- State $s_t = (x, y_{<t})$ = Prompt + Previously generated tokens.
- Action a_t = what agent does at t = Next token sampled/selected/generated
- Agent = Policy = $\pi_\theta(a_t \mid s_t)$ = Distribution over Actions (softmax over tokens)
- Trajectory = Full completion/response = $y = (y_1, y_2, \ldots, y_T)$
- Reward = Scalar feedback from $r_\psi(x,y)$
- Return = R_t = Total accumulated discounted reward from t onwards

- Sparse Reward = Reward only at final token
- Dense " = Per-step reward
- Value = $V_\phi(s_t)$ = Expected return from s_t = output of Value Model
- Advantage = A_t = [How much better a_t is compared to what was expected by V_ϕ] = computed from rewards & V_ϕ
- Policy Gradients = Updates to π_θ : $\pi_\theta^{new} \leftarrow \pi_\theta^{old} + \text{update}$

Base objective :

Let $x \sim D_p$ for which π_θ produces completion $y : \{y_1, \ldots, y_T\}$ & we have a reward model $r_\psi(x, y) \in \mathbb{R}$ s.t.

$$\max_\theta J_B(\theta) = \mathbb{E}_{\substack{x \sim D_p \\ y \sim \pi_\theta}} \left[r_\psi(x, y) \right] \quad \rightarrow ①$$

① Pushes π_θ towards sequences preferred by humans.

Regularize the base : Penalize deviations from π_{ref}

$$\max_\theta J_B(\theta) \quad \text{s.t.} \quad \mathbb{E}_{x \sim D_p} \left[KL(\pi_\theta \,||\, \pi_{ref}) \right] \leq \varepsilon$$

$$\Rightarrow \max_\theta J_B(\theta) - \beta \, \mathbb{E}_{x \sim D_p} \left[KL(\pi_\theta \,||\, \pi_{ref}) \right] \quad \rightarrow ②$$

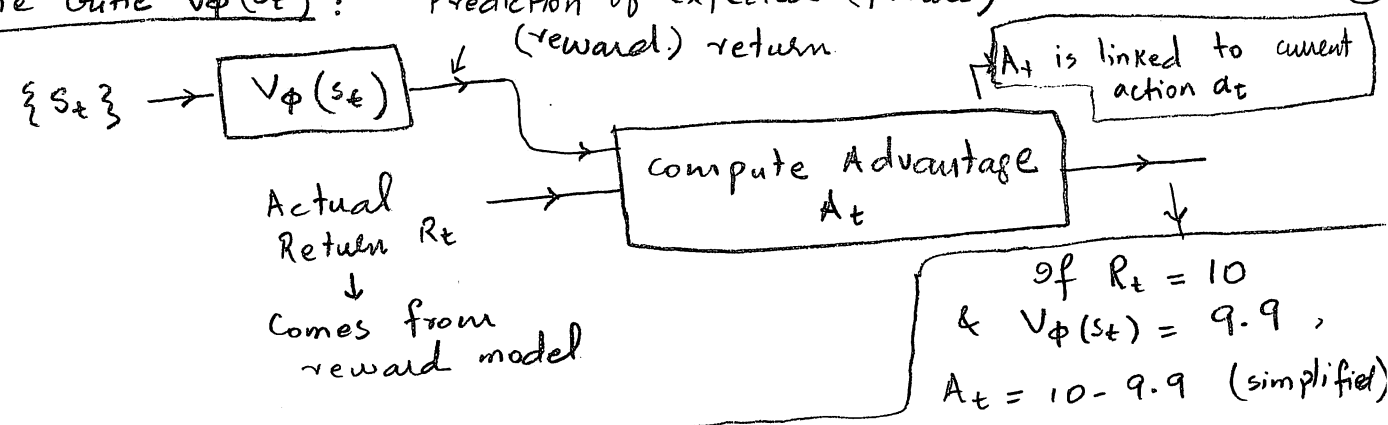
② avoids reward (model) hacking & also preserves SFT coverage (trust-region view).

Two issues : ① If any y gets high-reward (say = 10), we will keep generating that y ; however high-reward could be due to → π_θ or → Random chance or → RM's noise

② Reward = sparse ⇒ appears at <EOS>. If $T = 500$ tokens & $r = 10$ at end, which token (among 500) should get credit for ?

⇒ Early Tokens get neglected.

Bring in the Critic $V_\phi(s_t)$: Prediction of expected (future) (reward) return



If $R_t = 10$ & $V_\phi(s_t) = 9.9$, $A_t = 10 - 9.9$ (simplified)

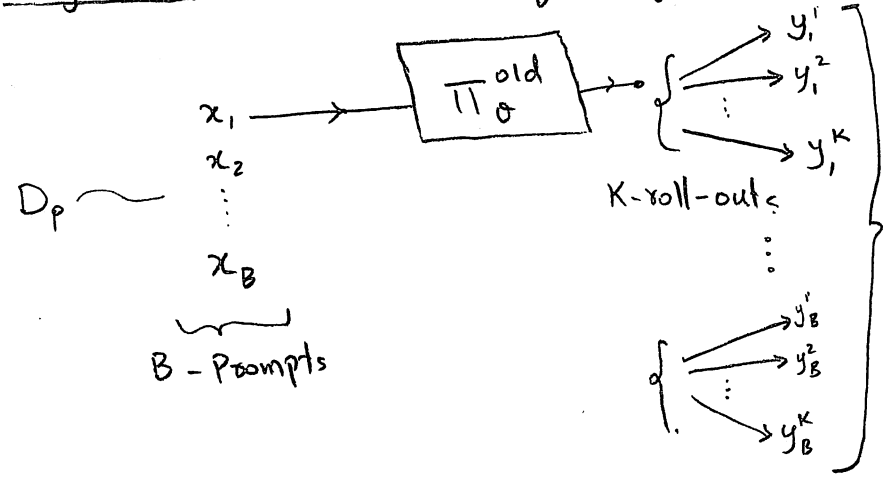
is low ⇒ a_t was good but not surprisingly good ⇒ π_θ does not change much ⇒ Reduces noise ⇒ Faster & stable training

Critic is trained using MSE:

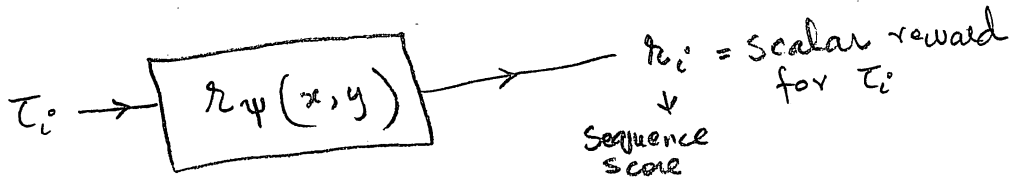
$$L_v(\phi) = E_t\left[\left(V_\phi(s_t) - R_t\right)^2\right]$$

Let's dig it deeper: we have the followings:

- Prompts per batch = B
  $(x_1, x_2, \ldots, x_B) \sim D_p$
- **Roll-outs:** = K → Each prompt x_b is fed to → π_θ^{old} K-times to get K completions for each x_b:
- **Trajectories:** Total no. of trajectories = N = K × B



- **Rewards (sparse):**



- **Cached Collection:** For each τ_i:
  - Prompt: x_i
  - Tokens: $y_{i,1:T_i}$
  - State: $s_{i,t} = (x_i, y_{i,\le t})$
  - $\log \pi_\theta^{old}(y_{i,t+1} | s_{i,t})$ — Softmax Distribution of prob.
  - $\log \pi_{ref}(y_{i,t+1} | s_{i,t})$ — " " "

With this set-up, we can have policy update at token-level.

Step-1 : Build Per-Token Reward = $r_{i,t}$:

Sparse trajectory reward = $r_i = r_{i,t}^{task} = \begin{cases} r_\psi(x_i, y_{i,1:T_i}) & t = T_i \\ 0 & t < T_i \end{cases}$

KL-shaping reward = $r_{i,t}^{KL} = -\beta \left[\log \pi_\theta^{old}(a_t|s_t) - \log \pi_{ref}(a_t|s_t) \right]$

(a_t: sampled token index at step t)

other dense terms (toxicity, length, etc.) = $r_{i,t}^{other}$

Total Per-step Reward = $r_{i,t} = r_{i,t}^{task} + r_{i,t}^{KL} + r_{i,t}^{other}$

Step-2 : choose value Targets (Ground Truth to train V_ϕ) :

$$L_V(\phi) = \frac{1}{\sum_i T_i} \sum_{i=1}^{N} \sum_{t=1}^{T_i} \left(V_\phi(s_{i,t}) - \underbrace{E\left[R_{i,t}^{target}\right]}_{= V_\phi^*(s_{i,t})} \right)^2$$

$R_{i,t}^{target}$ = The cumulated (discounted) sum of future rewards from s_t

$E\left[R_{i,t}^{target}\right]$ = Ground truth for $V_\phi(s_{i,t}) = V_\phi^*(s_{i,t})$

↳ Also called value function.

(E: over current policy randomness)

Some options to choose $R_{i,t}^{target}$:

① **Monte-Carlo (MC) Return :** $V_\phi^{MC}(s_{i,t}) \approx R_{i,t}^{target} = \sum_{u=t}^{T_i} \gamma^{u-t} r_{i,u}$

(single trajectory sample of the true expectation.)

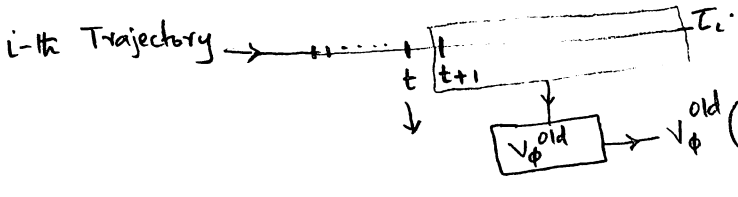
It has high variance as $R_{i,t}^{target}$ depends on all future randomness in $(r_{i,t+1}, \dots, r_{i,T})$

but no bias :

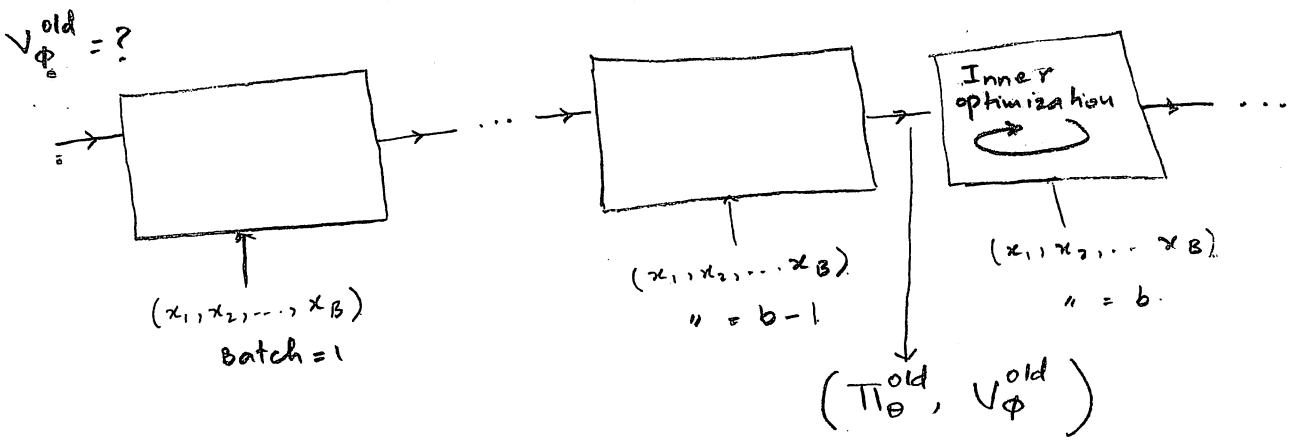
$E[V_\phi^{MC}] = V_\phi^*$

② **Temporal Difference : TD(0) :** $V_\phi^{TD}(s_{i,t}) = r_{i,t} + \gamma V_\phi^{old}(s_{i,t+1})$

(V_ϕ^{old}: Frozen snapshot of V_ϕ at last iteration.)



- It has low variance but biased if $E[V_\phi^{old}] \neq V_\phi^*$



$$V_\phi^{TD}(s_{i,t}) = r_{i,t} + \gamma V_\phi^{old}(s_{i,t+1})$$

↳ Also called bootstrap by one step.

③ GAE: Generalized Advantage Estimation: GAE combines best of both MC & TD(0): GAE is a geometrically weighted sum of TD errors: the instantaneous one-step TD error is given by:

$$\delta_{i,t} = r_{i,t} + \gamma V_\phi^{old}(s_{i,t+1}) - V_\phi^{old}(s_{i,t})$$

↓

TD error: if $r_{i,t}$ plus the next state's predicted value is much higher than the current state's predicted value, then δ_t is high ⇒ surprisingly good state transition $(s_t \to s_{t+1})$

$$A_{i,t}^{GAE(\lambda)} = \sum_{l=0}^{T_i - t} (\gamma\lambda)^l \delta_{i,t+l} \Rightarrow \boxed{V_\phi^{GAE}(s_{i,t+1}) = V_\phi^{old}(s_{i,t}) + A_{i,t}^{GAE(\lambda)}}$$

↓ GAE Advantage

↳ Also called λ-return.

We can show:

- $\lambda \to 1$, $V_\phi^{GAE} \to V_\phi^{MC}$: unbiased + high-variance
- $\lambda \to 0$, $V_\phi^{GAE} \to V_\phi^{TD}$: Biased + low-variance

⇒ $\boxed{0 < \lambda < 1}$ ✶ Trade-off control. ⇒ usually $\lambda \in [0.9, 0.98]$

Ex. Let N=1, T=3 with observed per-token reward: $r_1 = 0.05$, $r_2 = -0.02$, $r_3 = 1.6$

V_ϕ^{MC}: $R_3^{target} = r_3 = 1.6$, $R_2^{target} = r_2 + r_3 = 1.58$, $R_1^{target} = r_1 + r_2 + r_3 = 1.63$

$V_\phi^{MC}(s_3)$ ($\gamma = 1$) $V_\phi^{MC}(s_2)$ $V_\phi^{MC}(s_1)$

V_ϕ^{TD}: Let current critic snapshot predicts: $V_\phi^{old}(s_1) = 1.5$, $V_\phi^{old}(s_2) = 1.55$, $V_\phi^{old}(s_3) = 1.45$

$V_\phi^{TD}(s_1) = r_1 + V_\phi^{old}(s_2) = 0.05 + 1.55 = 1.6$

$V_\phi^{TD}(s_2) = r_2 + V_\phi^{old}(s_3) = -0.02 + 1.45 = 1.43$

$V_\phi^{TD}(s_3) = r_3 = 1.6$

V_ϕ^{GAE} : TD errors: $\delta_1 = r_1 + V_\phi^{old}(s_2) - V_\phi^{old}(s_1) = 0.05 + 1.55 - 1.5 = 0.1$ $(\gamma = 1)$

$\delta_2 = r_2 + V_\phi^{old}(s_3) - V_\phi^{old}(s_2) = -0.12$

$\delta_3 = r_3 - V_\phi^{old}(s_3) = 0.15$

GAE Advantage: $(\gamma = 1)$ let $\lambda = 0.9$: $A_3^{GAE} = \delta_3 = 0.15$

$A_2^{GAE} = \delta_2 + \lambda \delta_3 = 0.015$, $A_1'' = \delta_1 + \lambda \delta_2 + \lambda^2 \delta_3 = 0.1135$

λ-returns: $V_\phi^{GAE}(s_t) = V_\phi^{old}(s_t) + A_t^{GAE}$

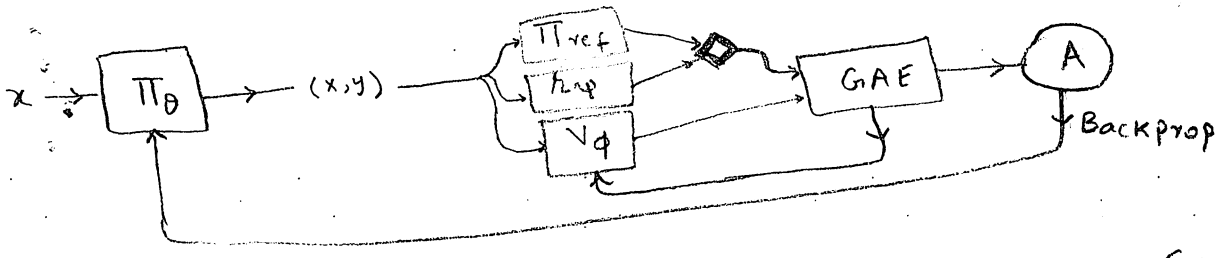
$\Rightarrow V_\phi^{GAE}(s_1) = 1.5 + 0.1135 = 1.6135 \rightarrow$ (bw MC 1.63 & TD(0) 1.6)

$V_\phi^{GAE}(s_2) = 1.55 + 0.015 = 1.565 \rightarrow$ (bw " 1.58 & " 1.43)

$V_\phi^{GAE}(s_3) = 1.45 + 0.15 = 1.6 \rightarrow$ should match MC at $t = T$

You can see: when $\lambda \rightarrow 1$, $V_\phi^{GAE} \rightarrow V_\phi^{MC}$

" $\lambda \rightarrow 0$, $V_\phi^{GAE} \rightarrow V_\phi^{TD(0)}$



Inputs:
- Collected trajectories: τ_i, $i=1,2,\dots,N$ where $\tau_i = \{(s_{i,t}, y_{i,t})\}$ generated from behavior policy π_θ^{old}. Completion y_i has seq. length $= T_i$
- Per token advantage: $A_{i,t}$, computed using frozen V_ϕ^{old}
- Cached log-probs.: $\log \pi_\theta^{old}(y_{i,t}|s_{i,t})$
- Reference log-probs.: $\log \pi_{ref}(y_{i,t}|s_{i,t})$

This means: $\pi_\theta^{old}(\text{next-token prob.} \mid s_{i,t})$ or $\pi_\theta^{old}(a_t|s_{i,t})$

- Advantage scores are standardized across all valid tokens in the batch as:

$$\tilde{A}_{i,t} = \frac{A_{i,t} - \mu_A}{\sigma_A}$$

- Critic targets: $V_\phi^{GAE}(s_{i,t})$
- $U = \sum_{i=1}^{N} T_i$ = Total valid tokens

Full-batch PPO:

For every token (i,t) in all dataset, we compute

$$\rho_{i,t}(\theta) = \exp\left(\log \pi_\theta(y_{i,t}|s_{i,t}) - \log \pi_\theta^{old}(y_{i,t}|s_{i,t})\right) = \frac{\pi_\theta(\cdot)}{\pi_\theta^{old}(\cdot)}$$

Prob. Ratio: Ratio of prob. of taking action a_t under the new policy π_θ versus the prob. under the old policy π_θ^{old} (π_θ^{old} generated completions).

Goal: If a_t was surprisingly good $\Rightarrow A_{i,t}^{GAE} > 0$, we want $\rho_{i,t}(\theta)$ to be high (increase prob. of a_t) & vice-versa.

$\Rightarrow \rho_{i,t} > 1 \Rightarrow$ current policy makes a_t more likely than before

$\rho_{i,t} < 1 \Rightarrow$ " " " less " " "

Whereas $\tilde{A}_{i,t} > 0 \Rightarrow a_t$ did better than expected $\Rightarrow$ increase its prob.

$\tilde{A}_{i,t} < 0 \Rightarrow$ " " worse " " $\Rightarrow$ decrease " "

Based on $\rho_{i,t}(\theta)$ & $A_{i,t}$, we can form the following objective to train our policy:

$$L^{CPI}(\theta) = E_t\left[\rho_{i,t}(\theta) \cdot A_{i,t}\right]$$ → Unclipped surrogate CPI (Conservative Policy Iteration)

$$L_{CPI}(\theta) = \frac{1}{U}\sum_{i=1}^{N}\sum_{t=1}^{T_i} \rho_{i,t}(\theta)\, \underbrace{A_{i,t}}_{\text{const.}}$$ → maximize it to get θ

- If $A_{i,t} > 0$; $\max L_{PG}(\theta) \Rightarrow$ Push $\rho_{i,t}$ ↑
- " " < 0; " " " " " ↓

- However, $\rho_{i,t}$ starts $\underbrace{\text{increasing}}_{\rho \to \infty}$ (when $A > 0$) or $\underbrace{\text{decreasing}}_{\rho \to 0}$ (when $A < 0$) without bound. optimizer will make very large policy jumps causing instability + overfitting + Distribution collapse.

⇒ We need a soft-trust-region around π_θ^{old}.

Trust-region PO (TRPO):

$$\max_\theta E_t\left[\rho_{i,t}(\theta) \cdot A_{i,t}\right] \text{ s.t.}$$

$$E_t\left[KL\left(\pi_\theta^{old}(\cdot|s_{i,t}) \,\|\, \pi_\theta(\cdot|s_{i,t})\right)\right] \le \delta$$

⇒ $$L_{TRPO}(\theta) = E_t\left[\rho_{i,t}(\theta) A_{i,t} - \beta\, KL\left(\pi_\theta^{old} \,\|\, \pi_\theta\right)\right]$$ → had to tune β

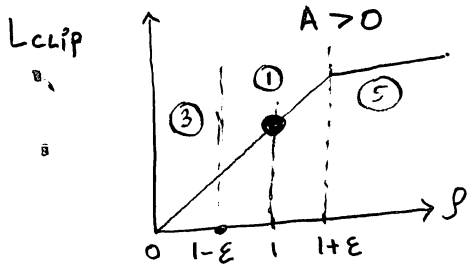
Proximal PO (PPO):

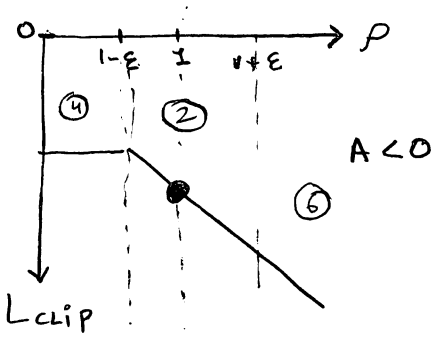
- Benefit is capped at $(1+\varepsilon)A$
- Harm " " " $(1-\varepsilon)A$

$$L_{clip}(\theta) = E_t\left[\min\left(\overbrace{\rho_{i,t} A_{i,t}}^{L_{CPI}},\ \overbrace{clip\left(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon\right) A_{i,t}}^{\text{modified } L_{CPI}}\right)\right]$$

- $clip(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)$ does not allow $\rho_{i,t}$ to go beyond region $[1-\varepsilon, 1+\varepsilon]$
- $\underbrace{\min}_{\text{worst case it is; to be cautious}}(a, b)$ = min of clipped & unclipped objectives ⇒ Final objective is lower-bound on unclipped objective
- when θ stays close to θ^{old} ⇒ $L_{clip}(\theta) = L_{CPI}(\theta)$.

As θ moves away from θ^{old}, L_{clip} & L_{CPI} are diff.





$\rho = 1$ = starting point of optimization

Region	$\rho_t > 0$	A_t	Return value of min(·)	clipped?	Sign of objective ↑	∇
①	$\rho_t \in [1-\varepsilon, 1+\varepsilon]$	+	$\rho_t(\theta) A_t$	No	+	✓
②	$\rho_t \in [1-\varepsilon, 1+\varepsilon]$	−	〃	〃	−	✓
③	$\rho_t < 1-\varepsilon$	+	〃	〃	+	✓
④	$\rho_t < 1-\varepsilon$	−	$(1-\varepsilon) A_t$	Yes	−	∇=0
⑤	$\rho_t > 1+\varepsilon$	+	$(1+\varepsilon) A_t$	Yes	+	∇=0
⑥	$\rho_t > 1+\varepsilon$	−	$\rho_t(\theta) A_t$	No	−	✓

Now, we can write complete loss as:

$$L_{PPO}(\theta, \phi) = -L_{CLIP}(\theta) + L_v(\phi) + L_{KL}(\theta) - L_{ENT}(\theta)$$

where $L_v(\phi) = \frac{1}{U} \sum_{i=1}^{N} \sum_{t=1}^{T_i} \left(V_\phi(s_{i,t}) - V_\phi^{GAE}(s_{i,t}) \right)^2$

These 2 are optional:

$$L_{KL}(\theta) = \frac{1}{U} \sum_{i,t} KL\left(\pi_\theta \,\|\, \pi_{ref}\right) \rightarrow \text{monitors per-token KL to SFT reference.}$$

$$L_{ENT}(\theta) = \frac{1}{U} \sum_{i,t} H\left(\pi_\theta(\cdot \mid s_{i,t})\right) \rightarrow \text{Discourages pre-mature collapse of token's distribution}$$

This is called RLHF with PPO which is a leading policy-gradient method; actor-critic policy gradient with a clipped trust region surrogate

- Note: $\frac{1}{U} \sum_{i=1}^{N} \sum_{t=1}^{T_i}$ → Full-batch PPO → Actually, we do mini-batches over a subset of trajectories → Sequence-minibatch PPO

Problems with PPO:

- V_ϕ can be inaccurate → destabilizes actor updates.
- Value-head + GAE + V_ϕ → more compute & memory.

Ex: x: "What is 2+2?"
LLM y: "The answer is ___"

- Get logits for last token produced by policy, apply softmax & then look-up probability for token "4" (correct answer)

Probability of "4"	π_θ^{old}	π_θ
	0.1	0.15

$$\Rightarrow \rho_t = \frac{0.15}{0.1} = 1.5$$

Compute raw reward:

Raw reward	V_ϕ Prediction (Expected reward)	Advantage
+1 (correct)	0.4	$1 - 0.4 = 0.6$
-1 (wrong)	0.2	$-1 - 0.2 = -1.2$

Raw Score: $\rho_t \times A_t = 1.5 \times 0.6 = 0.9$

Clip Ratio: $= clip(\rho_t A_t, 0.8, 1.2) = 1.2$

Clipped Score $= 1.2 \times 0.6 = 0.72$

Final Score: if $(A_t \geq 0)$ final_score $= \min(\text{Raw Score}, \text{clipped Score})$

$= 0.72$

else final-Score $= \max(\,,\,)$

loss: $loss = -\text{final_Score} + \beta \cdot KL(\pi_\theta \,\|\, \pi_{ref})$